04_ソースコード構成

04. ソースコード構成

1. ディレクトリツリー

```
gym-checkin/
├── api/                          # Vercel Serverless Functions
（12 個上限）
│   ├── admin.ts                  # 管理者 API（action で分岐）
│   ├── checkins/
│   │   ├── index.ts              # 予約作成／一覧／空き状況／イベント・
スクール
│   │   └── [id].ts               # 予約詳細／キャンセル／領収書 PDF
│   ├── coupons/validate.ts
│   ├── member-types/index.ts
│   ├── payments/
│   │   ├── create-checkout.ts    # Stripe セッション作成
│   │   ├── stripe-callback.ts    # 決済成功コールバック
│   │   └── stripe-webhook.ts     # Stripe Webhook（署名検証）
│   ├── prices/calculate.ts
│   ├── reviews/index.ts
│   └── users/
│       ├── me.ts
│       └── membership.ts
│
├── server-lib/                   # サーバー側共通ライブラリ
│   ├── auth.ts                   # LIFF accessToken 検証
│   ├── firebase.ts               # Firestore 初期化／COLLECTIONS
│   ├── line.ts                   # LINE Messaging API ラッパ
│   ├── pdf.ts                    # 領収書 PDF 生成（pdf-lib）
│   ├── remotelock.ts             # RemoteLock OAuth / Booking
│   └── logger.ts                 # 構造化ログ（JSON 1 行／Vercel ロ
グ向け）
│
├── src/                          # フロントエンド（React）
│   ├── App.tsx                   # ルータ・LIFF 初期化
│   ├── main.tsx
│   ├── components/common/        # 共通コンポーネント（Loading,
```

```
DebugPanel, 等)
│   ├── features/
│   │   ├── location/             # 拠点選択
│   │   ├── home/                 # 施設選択
│   │   ├── checkin/              # 単日／複数日／定期予約
│   │   ├── payment/              # 確認・クーポン・決済
│   │   ├── complete/             # 完了・PIN 表示
│   │   ├── review/               # レビュー投稿
│   │   ├── reservations/         # 予約一覧（自分）
│   │   ├── events/               # イベント一覧／詳細
│   │   ├── schools/              # スクール一覧／詳細
│   │   ├── admin/                # 管理画面（AdminPage,
CalendarTab)
│   │   └── legal/                # プライバシー／特商法
│   ├── lib/
│   │   ├── api.ts                # API クライアント + 型
│   │   ├── gcal.ts               # Google Calendar deeplink
│   │   ├── liff.ts               # LIFF SDK ラッパ
│   │   ├── locations.ts          # 拠点メタデータ
│   │   ├── price.ts              # フロント側の料金計算
│   │   └── logger.ts             # ブラウザログ
│   ├── stores/
│   │   ├── checkinStore.ts       # 予約フォームの状態
│   │   └── debugStore.ts         # 開発用スキップフラグ
│   ├── styles/
│   └── index.css
│
├── public/                       # 静的アセット
├── dist/                         # ビルド生成物（git 管理外）
├── Document/                     # 【本フォルダ】設計書・マニュアル
├── docs/                         # 画面仕様（旧）
├── checkin/                      # 画面キャプチャ
├── SmartLockディバイス情報/      # RemoteLock 台帳
├── paypay有効化/                 # PayPay 有効化 PDF
├── みんなの体育館数値取得画像/    # 売上参考資料
├── index.html                    # Vite エントリ
├── vite.config.ts
├── vercel.json                   # Vercel ビルド／CORS／SPA Rewrite
├── tailwind.config.js
├── postcss.config.js
├── eslint.config.js
├── tsconfig*.json
├── package.json / package-lock.json
├── README.md / 運用マニュアル.md / KEYS_MEMO.md
└── .env / .env.vercel            # 環境変数（git 管理外）
```

---

2. 主要モジュール解説

2.1 `src/App.tsx`

- LIFF 初期化（`initializeLiff`）→ `userApi.getMe()` で Firestore に自動登録
- `BrowserRouter` の下に全ルートを定義
- `DebugPanel` を常時マウント（本番でも長押しで開く開発用）
- 起動時に `logger.op('app.boot')` を出力

2.2 `src/lib/liff.ts`

- 開発モード `import.meta.env.DEV && !VITE_LIFF_ID` のときはモックプロファイルで継続
- 本番で `VITE_LIFF_ID` 未設定のときはエラー
- `getAccessToken()` が Bearer として送信される

2.3 `src/lib/api.ts`

- `fetch` ベースの薄いクライアント
- ドメイン別に `userApi` / `checkinApi` / `priceApi` / `couponApi` / `memberTypeApi` / `membershipApi` / `reviewApi` / `eventApi` / `schoolApi` / `announcementApi` / `adminApi` を公開
- 型 `LocationId` / `FacilityType` / `CheckinStatus` / `Checkin` / `Coupon` / `MemberType` / … を共通化

2.4 `src/stores/checkinStore.ts`

- Zustand の単一 store（永続化なし）
- 単日・複数日・定期予約の状態をまとめて保持
- `setMultiDateMode(false)` で定期予約情報もクリア

2.5 `src/stores/debugStore.ts`

- `paymentEnabled` / `remoteLockEnabled` の 2 フラグを localStorage に保存
- Payment／RemoteLock をスキップしてテストできるトグル

2.6 `server-lib/firebase.ts`

- 環境変数 `FIREBASE_SERVICE_ACCOUNT_KEY`（Base64 JSON）を復号して `firebase-admin` を初期化
- `COLLECTIONS` 定数で 12 コレクション名を集約

2.7 `server-lib/auth.ts`

- `Authorization: Bearer <token>` を取り出し、`https://api.line.me/v2/profile` で検証
- 開発環境では `mock-access-token-for-development` を許容

2.8 `server-lib/remotelock.ts`

- OAuth2 Client Credentials → 5 分マージンで accessToken をキャッシュ
- `getDeviceIds(location, facilityType)` で対象デバイスを決定
  - ASP: `ENTRANCE` + (`GYM` or `TRAINING`)
  - YABASE: `YABASE` 1 台のみ
- `createBooking({ pin })` でグループ予約時に PIN を引き継げる
- `cancelBooking(checkinId, deviceCount)` でキャンセル

2.9 `server-lib/pdf.ts`

- `pdf-lib` + `@pdf-lib/fontkit` で日本語 TTF を埋め込み領収書 PDF を生成
- `/api/checkins/[id]?format=receipt` / `/api/admin?action=receipt` から呼び出す

2.10 ログ (`src/lib/logger.ts` / `server-lib/logger.ts`)

- フロント: コンソール向け読みやすい整形（`[LEVEL] HH:mm:ss.SSS [scope] ...`）
- サーバー: 1 行 JSON（Vercel ログの検索に最適）
- 両者とも `createLogger(scope)` → `{ debug, info, warn, error, op }`
- 操作ログは `op(action, payload)` を必ず使う

2.11 `src/components/common/DebugPanel.tsx`

- `debugStore` のフラグを UI で切り替える開発支援パネル
- 本番でも HTML 内に存在するが、トリガーを長押しにしているため通常利用者には見えない

---

3. ログを差し込む方針

3.1 フロント

- 画面の `useEffect` に `log.op('<page>.view')` を入れる（必要に応じ拡張）
- ボタン操作は `onClick` 内で `log.op('<page>.<action>', { ...payload })`
- エラー境界・catch 句に `log.error`

3.2 サーバー

- 各ハンドラの冒頭で `log.debug('request.in', { method, query })`
- 認可失敗: `log.warn('unauthorized')`
- 成功の監査ログ: `log.op('<domain>.<action>', { lineUserId, ... })`
- 例外: `log.error('handler.fail', { message: String(error) })`

---

4. ビルド／実行コマンド

```
npm run dev        # Vite dev server (LIFF モック or 実 LIFF)
npm run build      # tsc -b && vite build
npm run preview    # ビルド物を起動
npm run lint       # ESLint
```

Vercel 環境では `vercel.json` の `buildCommand` が実行される。